Final Project Report

Metro Ticket Generating System in ServiceNow | Capstone report | 11 October 2026

This report follows the supplied Final Report Template and presents the Skill Wallet metro ticket capstone as a ServiceNow catalog implementation. It brings together the problem, requirements, design, configured workflow, evidence, and conclusion.

1 Introduction

1.1 Project Overview

Metro Ticket Generating System in ServiceNow is a catalog-based digital ticketing demonstration. A passenger chooses two stations, journey type, passenger count, and payment mode in Service Portal. Client logic displays a fare, and a Service Portal widget presents a QR image generated from the exercise ticket URL.

1.2 Purpose

The project explores how a familiar ServiceNow service request interface can replace a paper-oriented first step in metro ticketing. It emphasizes accessible route entry, repeatable fare logic, reusable reference data, and a QR presentation pattern.

2 Ideation Phase

2.1 Problem Statement

A commuter needs a clear way to choose a route and see a ticket amount without relying on a counter. Operations staff need consistent station names and maintainable fare logic. The project addresses this with a Service Catalog item and centrally maintained station reference table.

2.2 Empathy Map

Commuter view	Design response
Wants a fast, simple booking path	Single catalog form with ordered fields.
Needs confidence in the selected journey	Required source and destination references.
Wants to understand the amount	Read only fare fields calculated from route and count.
Needs a digital artifact	QR image presented by a dedicated widget.

2.3 Brainstorming

The Skill Wallet roadmap compares station mapping, fare automation, QR display, portal navigation, and user acceptance testing as the core capabilities. The implementation uses built-in catalog records and a small amount of client scripting.

3 Requirement Analysis

3.1 Customer Journey

Home > Service Catalog > Services > Book a Ticket > select stations > choose journey and passenger count > review fare > choose payment mode > view QR preview.

3.2 Solution Requirements

- ServiceNow Personal Developer Instance and Service Portal.
- Custom station table with Station Name field and station records.
- Catalog item with eight ordered variables.
- Fare onChange script, payment UI policy, QR onSubmit script, and Metro QR Widget.
- Browser-based functional checks and screenshots for the project demo.

3.3 Data Flow

Passenger input enters catalog variables. Station references resolve to u_metro_station_details records. The fare script reads source, destination, journey type, and passenger count; it maps the route to a fare and writes the appropriate display field. QRGeneration passes an image URL to metro_qr_widget through spModal.

3.4 Technology Stack

Technology	Use
ServiceNow Service Catalog	Catalog item and variables
Service Portal	Passenger-facing interface
Catalog Client Scripts	Fare and QR presentation logic
Catalog UI Policy	Payment field visibility
Custom table	Station reference data
Service Portal Widget	QR image display

4 Project Design

4.1 Problem Solution Fit

The catalog form gives one entry point for route and fare choices. Station references reduce free-text variation. A separate QR widget keeps presentation logic outside the booking form.

4.2 Proposed Solution

A requester opens Book a Ticket in Service Portal, chooses stations and journey details, sees a calculated amount, and opens a QR preview. An administrator maintains the station records, scripts, UI policy, and widget.

4.3 Solution Architecture

Service Portal -> Book a Ticket variables -> Metro Station Details references -> Fare auto Calculation and Fields Visibility -> QRGeneration -> spModal -> Metro QR Widget. The detailed record mapping and code are in the Technical Blueprint.

5 Project Planning and Scheduling

Milestone	Output
Requirement analysis	Objectives, scope, stakeholders, roadmap
Backend and configuration	Station table and Station Name field
UI and automation	Catalog item, variables, fare script, UI policy, QR widget
Testing and security	Portal checks, station access review, UAT plan
Documentation and presentation	Manuals, blueprint, report, demo plan, conclusion

6 Functional and Performance Testing

The project test plan covers required station inputs, reference lookups, sample single and return fare paths, payment visibility, QR preview, and role access. Performance measurements should record actual load and calculation times on the same browser and network. The Test Plan and UAT Record supplies the case list and result sheet.

7 Results

The observed configuration contains an active Book a Ticket catalog item with eight variables, a 27-record station table, one catalog UI policy, two active catalog client scripts, and the Metro QR Widget. A read-only portal check displayed 11 for Miyapur to JNTU College with Single Journey and one passenger. The configured onSubmit path presents the exercise QR modal and cancels request submission.

8 Advantages and Disadvantages

Advantages	Disadvantages or trade-offs
Uses built-in ServiceNow catalog and portal components.	A user needs access to the configured ServiceNow instance.
Station names are centralized in a reusable table.	The fare matrix is maintained inside a client script.
QR rendering is isolated in a reusable widget.	The exercise QR URL and external image endpoint are demonstration components, not a payment or gate service.

9 Conclusion

The capstone demonstrates a ServiceNow-based metro booking form with reference station data, script-driven fare display, conditional payment input, and widget-based QR presentation. The components provide a coherent prototype for communicating the passenger journey and the corresponding administrator configuration.

10 Future Scope

Future work can connect a real ticket record to the QR payload, add email or SMS delivery, integrate a payment gateway, support station authority validation, and build travel analytics. These extensions should be tested with real request records and role-specific access.

11 Appendix

Artifact	Location
Source code snapshots	5. Project Development Phase
Functional specification	6. Project Documentation
Technical blueprint	3. Project Design Phase / Solution Architecture
Setup manual	6. Project Documentation
UAT record	5. Project Development Phase / User Acceptance Testing
Demo plan	7. Project Demonstration
Project conclusion	6. Project Documentation